

TEST MEMORIA+ RAZONAMIENTO (QWEN)

Aquí tienes 3 tests diseñados específicamente para validar el ciclo completo de Memoria + Razonamiento + Guardrails de la solución que te entregué. Están estructurados para ejecutarse vía CLI, MCP o directamente con las clases Python. Cada test incluye el prompt/comando, el flujo de ejecución y cómo validar el resultado.

TEST 1: Persistencia y Recuperación de Memoria

Objetivo: Verificar que `architect_save` almacena correctamente metadatos enriquecidos y que `architect_search` lo recupera con alta relevancia.

♦ Comando / Prompt

```
bash
# 1. Guardar decisión
architect_save \
  --query "Cómo manejar rate limits en Express" \
  --response "Usar express-rate-limit con ventana de 15min y 100 req/IP. Fallback a Redis si escalamos." \
  --type decision \
  --why "Evitar timeouts en API pública sin bloquear tráfico legítimo" \
  --tags api,express,seguridad

# 2. Recuperar inmediatamente
architect_search --query "rate limits express producción" --limit 3
```

Comportamiento Esperado

- `save` retorna: `{"status": "saved", "id": "mem_XXXXX...", "memory": {...}}`
- `search` retorna una lista donde el primer elemento contiene:
 - `title`: coincide con el query guardado
 - `what`: resume la respuesta exacta
 - `relevance`: > 0.75
 - `tags`: incluye `api`, `express`, `seguridad`
 - `type`: `decision`

Cómo Validar

```
python
```

```
# Validación rápida en Python (usa las clases del código entregado)
from core.memory import StructuredMemory
mem = StructuredMemory()
res = mem.search("rate limits express producción", limit=1)
assert res[0]["type"] == "decision"
assert "express-rate-limit" in res[0]["what"]
assert res[0]["relevance"] > 0.7
print("✅ TEST 1 PASSED")
```

TEST 2: Razonamiento (CoT) + Inyección de Contexto

Objetivo: Verificar que `architect_generate` con `use_cot=true` usa la memoria guardada para razonar paso a paso, cita fuentes y no alucina.

- ♦ Comando / Prompt

```
bash
```

```
architect_generate \
```

```
--query "¿Cómo implementar rate limiting seguro sin afectar usuarios legítimos?" \
```

```
--use_cot true
```

Comportamiento Esperado

- La respuesta sigue estrictamente la estructura:

Thought: [análisis inicial]

Action: [paso concreto]

Observation: [resultado]

...

Final Answer: [conclusión clara con citas [1]]

- Incluye referencia explícita a la memoria guardada en Test 1 (ej: `[1]` o `"según nuestra decisión de arquitectura..."`)
- No inventa librerías, versiones ni comportamientos no documentados en `context.md` o memoria.

✓ Cómo Validar

python

```
from core.pipelines import LangChainPipelines
from core.router import FreeLLMRouter
from core.memory import StructuredMemory

router = FreeLLMRouter()
mem = StructuredMemory()
pipelines = LangChainPipelines(router, mem)

reply = pipelines.cot_reasoning("¿Cómo implementar rate limiting seguro sin afectar usuarios legítimos?")
assert "Thought:" in reply
assert "Final Answer:" in reply
assert "[1]" in reply or "rate" in reply.lower()
print("✓ TEST 2 PASSED")
```

TEST 3: Guardrails, Verificación y Detección de Conflictos

Objetivo: Verificar que el sistema detecta contradicciones al guardar memoria nueva y que `architect_verify` rechaza afirmaciones no respaldadas.

♦ Comando / Prompt

bash

```
# 1. Intentar guardar decisión contradictoria
architect_save \
  --query "rate limits en producción" \
  --response "Desactivar rate limit completamente en producción para maximizar rendimiento" \
  --type decision \
  --why "Mejor latencia percibida" \
  --tags api,optimización
```

```
# 2. Verificar una afirmación falsa contra la memoria existente
```

```
architect_verify \
```

```
--context "Usar express-rate-limit con ventana de 15min y 100 req/IP. Fallback a Redis si escalamos." \
```

```
--claim "El límite recomendado es 1000 req/min sin necesidad de Redis"
```



Comportamiento Esperado

- `save` retorna: `{"status": "conflict_detected", "conflicts": [...], "message": "⚠ Se detectaron X posibles contradicciones"}`
- `verify` retorna: `{"valid": false, "reason": "..."} o baja confianza (< 0.65)`
- El sistema no sobrescribe la memoria original y sugiere revisión.



Cómo Validar

```
python
```

```
# Test de conflicto
```

```
mem = StructuredMemory()
```

```
conflict_res = mem.save(  
    query="rate limits en producción",  
    response="Desactivar rate limit completamente...",  
    memory_type="decision",  
    why="Mejor latencia percibida",  
    tags=["api", "optimización"],  
    check_conflicts=True  
)
```

```
assert conflict_res["status"] == "conflict_detected"
```

```
# Test de verificación
```

```
pipelines.verify(  
    context="Usar express-rate-limit con ventana de 15min y 100 req/IP...",  
    reply="El límite recomendado es 1000 req/min sin necesidad de Redis"
```

```
) == False
```

```
print("✅ TEST 3 PASSED")
```



Script Unificado de Ejecución Rápida

Guarda esto como `run_memory_tests.py` en la raíz del proyecto y ejecútalo:

```
python
```

```
#!/usr/bin/env python3
"""Suite exprés para validar Memoria + Razonamiento + Guardrails"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))

from core.memory import StructuredMemory
from core.pipelines import LangChainPipelines
from core.router import FreeLLMRouter

def test_suite():
    print("🔍 Iniciando suite de validación...")
    router = FreeLLMRouter()
    mem = StructuredMemory()
    pipes = LangChainPipelines(router, mem)

    # TEST 1
    print("\n📦 Test 1: Persistencia...")
    mem.save("Cómo manejar rate limits", "Usar express-rate-limit 15min/100req...",
"decision", "Evitar timeouts", tags=["api"])
    res = mem.search("rate limits express", limit=1)
    assert res and "express-rate-limit" in res[0]["what"]
    print("✅ PASSED")

    # TEST 2
    print("\n🧠 Test 2: Razonamiento CoT...")
    reply = pipes.cot_reasoning("¿Cómo aplicar rate limiting sin bloquear tráfico real?")
    assert "Thought:" in reply and "Final Answer:" in reply
    print("✅ PASSED")

    # TEST 3
    print("\n🛡️ Test 3: Conflictos + Verificación...")
    c = mem.save("rate limits", "Desactivar rate limit en prod...", "decision", "mejor latencia", tags=["api"])
    assert c["status"] == "conflict_detected"
    print("✅ PASSED")

    print("\n🎉 TODOS LOS TESTS PASARON. El ciclo Memoria→Razonamiento→Guardrails funciona correctamente.")

if __name__ == "__main__":
    try:
        test_suite()
    except AssertionError as e:
```

```
print(f"\n❌ FALLO EN VALIDACIÓN: {e}")
sys.exit(1)
except Exception as e:
    print(f"\n⚠️ ERROR TÉCNICO: {e}")
    sys.exit(2)
```

▶️ Cómo ejecutar

```
bash
```

```
python run_memory_tests.py
```

📋 Checklist de Interpretación

Resultado	Qué significa	Acción
✅ PASSED en los 3	El motor guarda, razona con contexto y aplica guardrails correctamente.	Listo para producción.
❌ FALLO en Test 1	ChromaDB no persiste o embeddings fallan.	Verifica <code>chromadb>=0.4.22</code> , permisos en <code>.ai_memory/</code> , y que <code>all-MiniLM-L6-v2</code> se descarga.
❌ FALLO en Test 2	El LLM no sigue CoT o no inyecta memoria.	Revisa <code>COT_PROMPT</code> en <code>pipelines.py</code> y que <code>router.get_model()</code> devuelve un modelo activo.
❌ FALLO en Test 3	No detecta conflictos o verifica mal.	Ajusta <code>threshold=0.75</code> en <code>_detect_conflicts</code> o verifica que <code>VERIFY_PROMPT</code> fuerza respuesta <code>VALID/INVALID</code> .

¿Quieres que te genere:

1.  Un `pytest` completo con fixtures y mocks para CI/CD?
2.  Un script que ejecute estos tests vía MCP tool calls (simulando cómo lo haría Claude Code / OpenCode)?
3.  Un dashboard de métricas (hit-rate, precisión@k, tasa de conflictos) para monitoreo en producción?